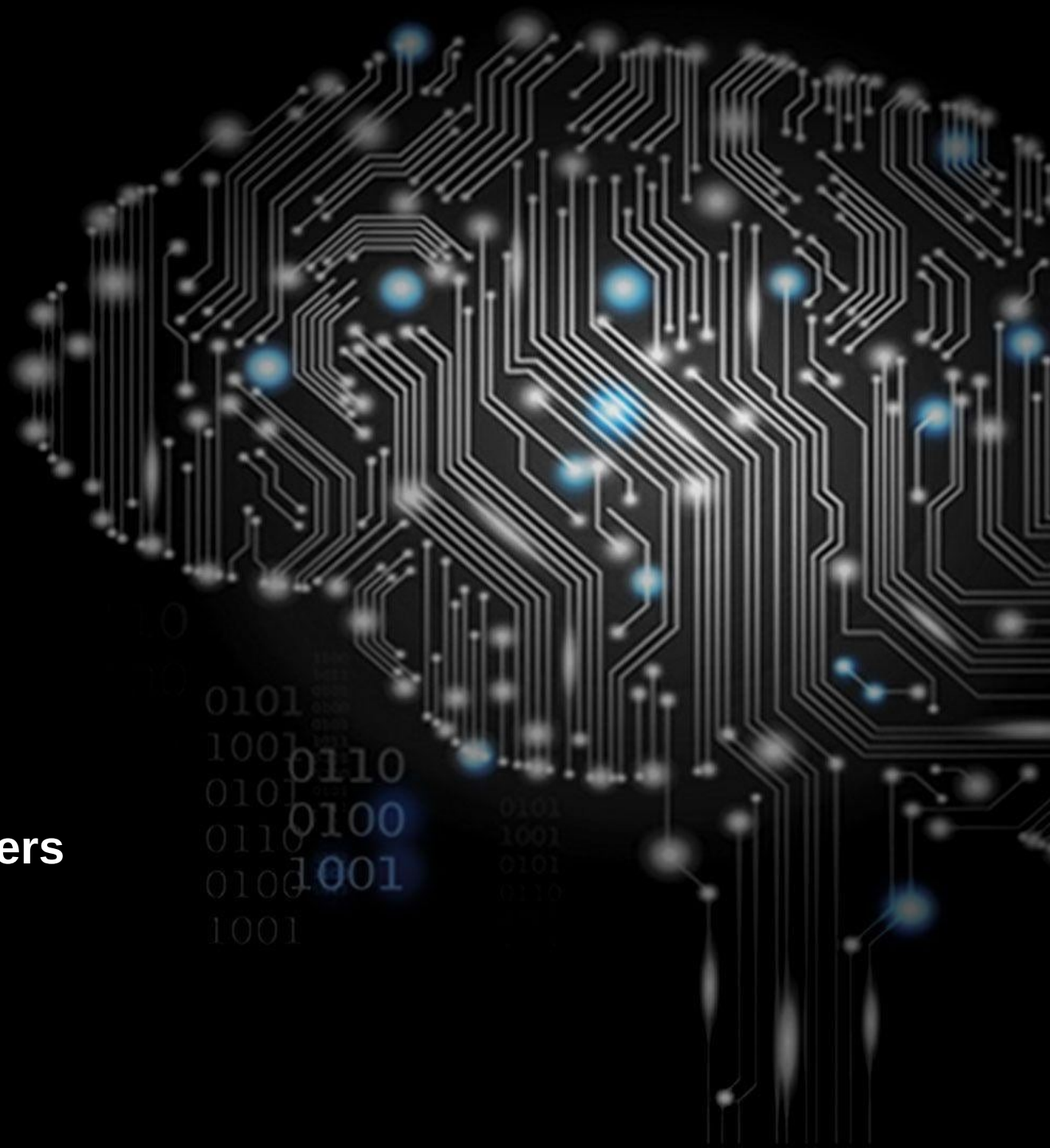


Introduction to Generative AI for Developers

1. GenAI Foundations for Java Developers

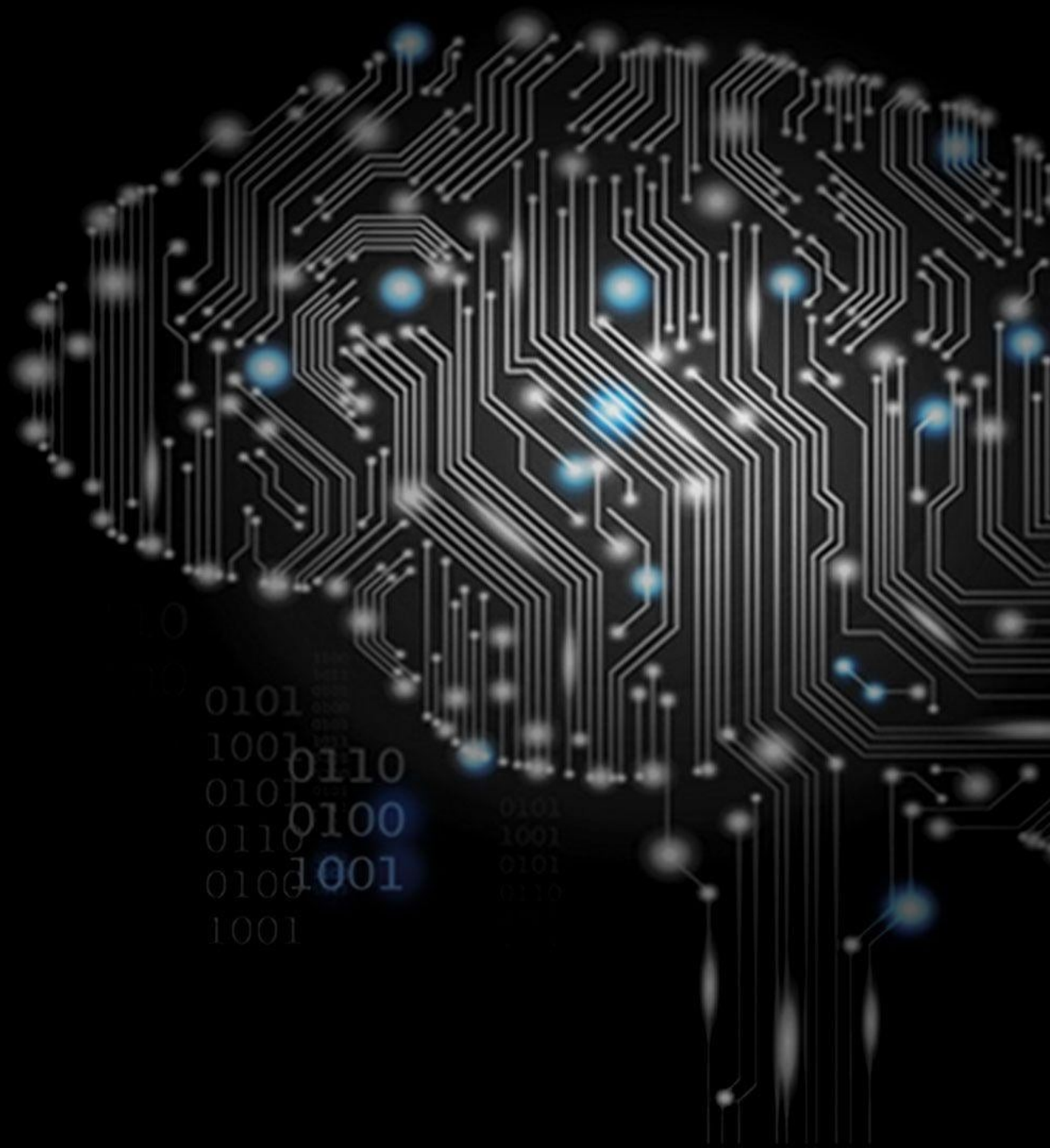
Sections

- How LLMs work from a dev perspective
- Reasoning models and generation controls
- Cost, latency and quality
- The model landscape and model hosting
- The Java ecosystem



GenAI Foundations for Java Developers

1. How LLMs work from a dev perspective

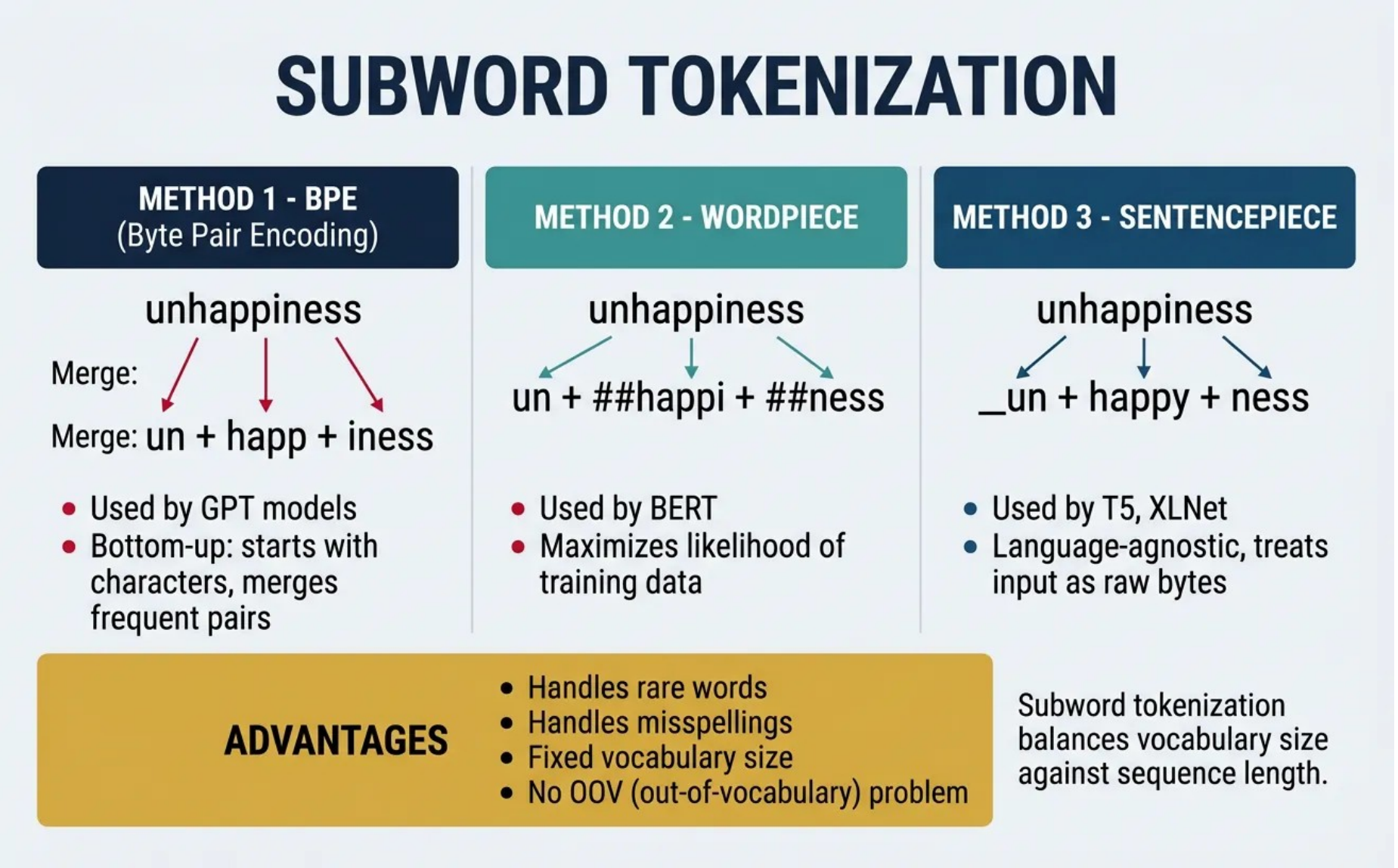


Tokens as basic units

- Models read and write tokens
  - Sub-word fragments produced by a tokenizer, which rarely line up exactly with characters or words
    - *Common English words are often one token*
    - *Rarw are words, identifiers and numbers split into several*
  - A common rule of thumb for English is roughly four characters per token
  - Tokens are the unit of billing, rate limits, context size and latency
    - *Input tokens and output tokens are priced separately; output is typically several times more expensive*
  - Token counts vary by language and content type
    - *French and other non-English text generally needs more tokens per word than English, which matters for a bilingual applications*
    - *JSON, XML and source code are token-heavy because of punctuation and whitespace*
  - Each provider uses its own tokenizer, so the same prompt has a different token count on different models



Tokens as basic units



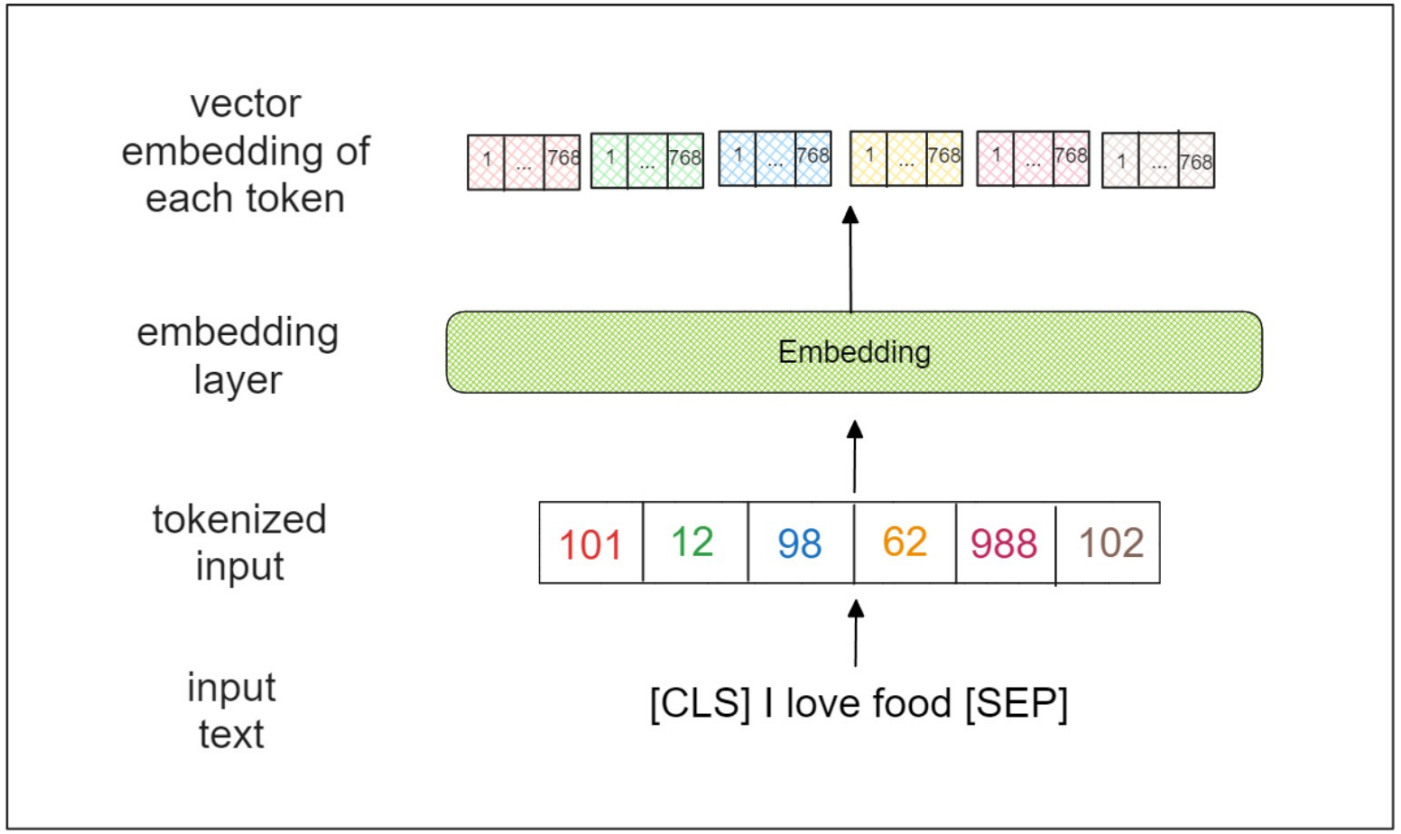
https://www.wasilzafar.com/pages/series/nlp/nlp-tokenization-text-cleaning.html

From tokens to vectors

- Inside the model, each token ID is converted into a vector
  - Current models use vectors with thousands of dimensions per token
  - Vectors place tokens in a high-dimensional space where related meanings sit close together
    - *Early word-embedding research showed that directions in this space capture relationships; the classic example is that the vector for "king" minus "man" plus "woman" lands near "queen"*
  - Transformer layers then adjust each token's vector based on the surrounding tokens, a mechanism called attention
    - *The result is contextual: "bank" in "river bank" and "bank" in "bank account" end up with different vectors*
  - The final layer converts the last vector into a probability for every token in the vocabulary, and the next token is chosen from that distribution
  - Developers never handle these internal vectors directly
    - *But they explain why models cope well with paraphrases, synonyms and spelling variation*



From tokens to vectors



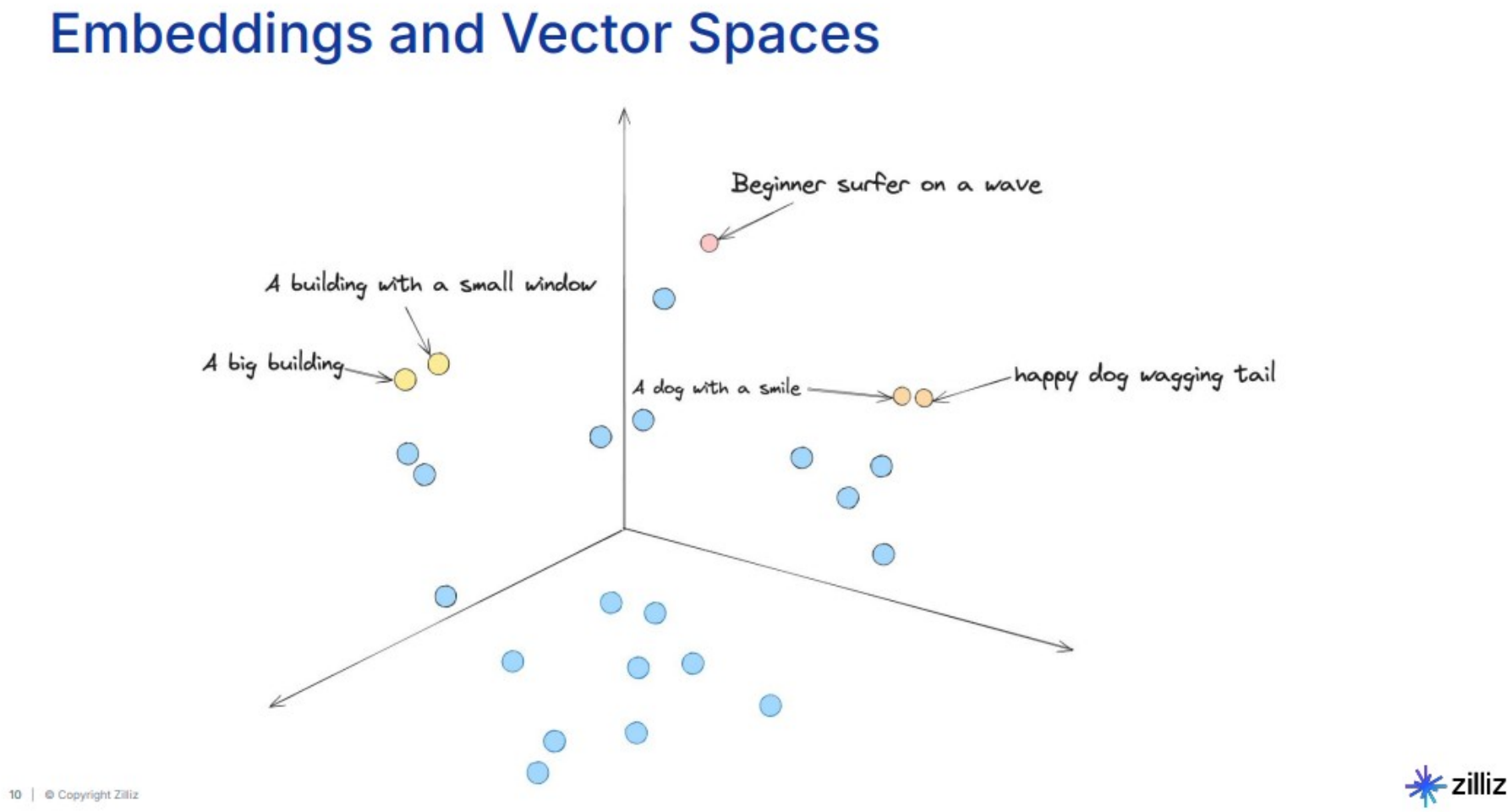
https://zilliz.com/learn/transforming-text-the-rise-of-sentence-transformers-in-nlp

Embedding models

- Separate, smaller models that turn a whole piece of text into one fixed-length vector
  - Texts with similar meaning produce vectors that are close together, usually measured with cosine similarity
    - *"How do I reset my online banking password?" and "I'm locked out of my account" score as similar despite sharing almost no words*
- Common uses in a bank
  - Semantic search over policies and procedures, and retrieval for RAG
  - Clustering and classifying complaints, detecting duplicates, flagging unusual messages
- Engineering considerations
  - Vectors from different embedding models cannot be compared; changing the model means re-embedding every stored document
    - *Store the embedding model identifier with each vector*
  - Treat embeddings of customer data as sensitive
    - *Research has shown that much of the original text can be recovered from its embedding*
- In Java
  - Spring AI and LangChain4j both provide an EmbeddingModel abstraction and vector store integrations



From tokens to vectors



https://zilliz.com/blog/producing-structured-outputs-from-llms-with-constrained-sampling

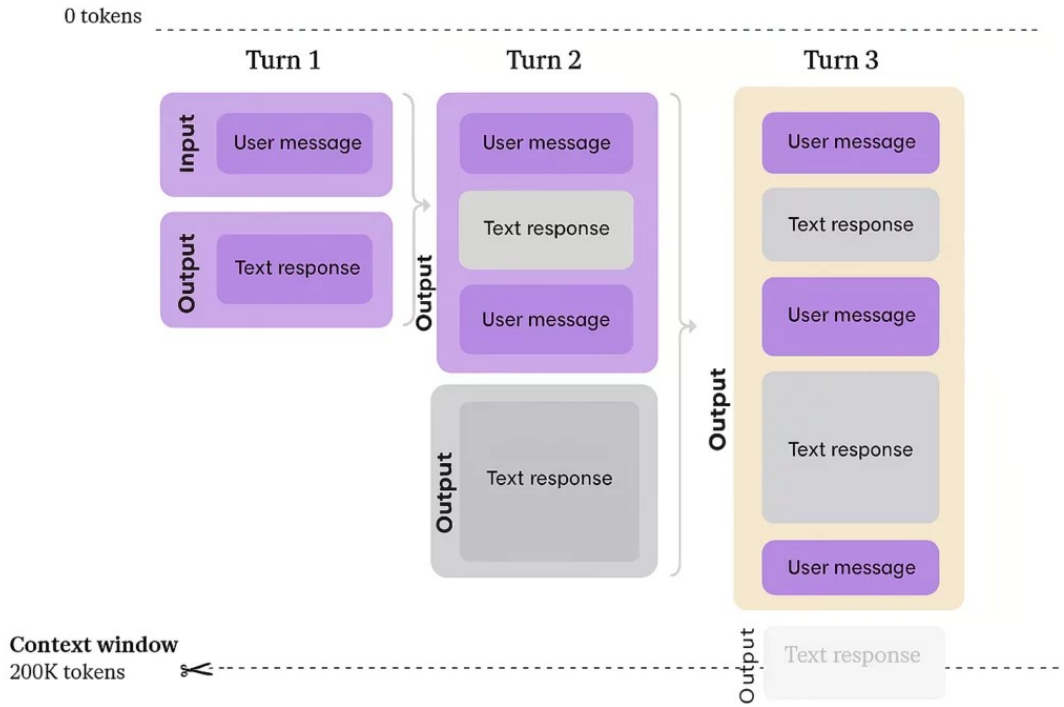
Context windows

- The maximum number of tokens a model can process in one request
  - Counting input, reasoning and output together
    - *Current frontier models accept hundreds of thousands of tokens; some offer one million*
- Everything the model "knows" about the conversation must be in the context on every API call
  - The API is stateless
    - *unless you use a provider feature that stores conversation state; prior turns are resent and re-billed*
  - A larger window is not the same as reliable use of the window
    - *Models retrieve information placed in the middle of long contexts less reliably than information at the start or end*
    - *Long contexts raise cost and latency on every call*
  - Design implication
    - *Supply only the context relevant to the task, since more context is not automatically better*



Context windows

- Analogy
  - The context window behaves like a fixed-size request buffer that is rebuilt on every call
  - There is no server-side session holding the conversation unless you opt into one.
- Some pricing tiers change above a threshold
  - For example, OpenAI lists GPT-6.1 Sol pricing "up to 272K input"
    - *So very long prompts can also change the unit price.*



https://www.allaboutai.com/ai-glossary/context-window/

Next-token prediction and non-determinism

- An LLM generates text one token at a time by producing a probability distribution over possible next tokens and selecting one
- Output varies between identical requests for two distinct reasons
  - Sampling: the model deliberately draws from the distribution rather than always choosing the most likely token
  - Inference infrastructure: even with sampling minimized, results can differ because servers batch many users' requests together
    - *Research published in 2025 identified batch-size dependence in GPU kernels as a primary cause of run-to-run variation*
- Engineering consequence
  - Treat every LLM call as non-deterministic, even with fixed settings



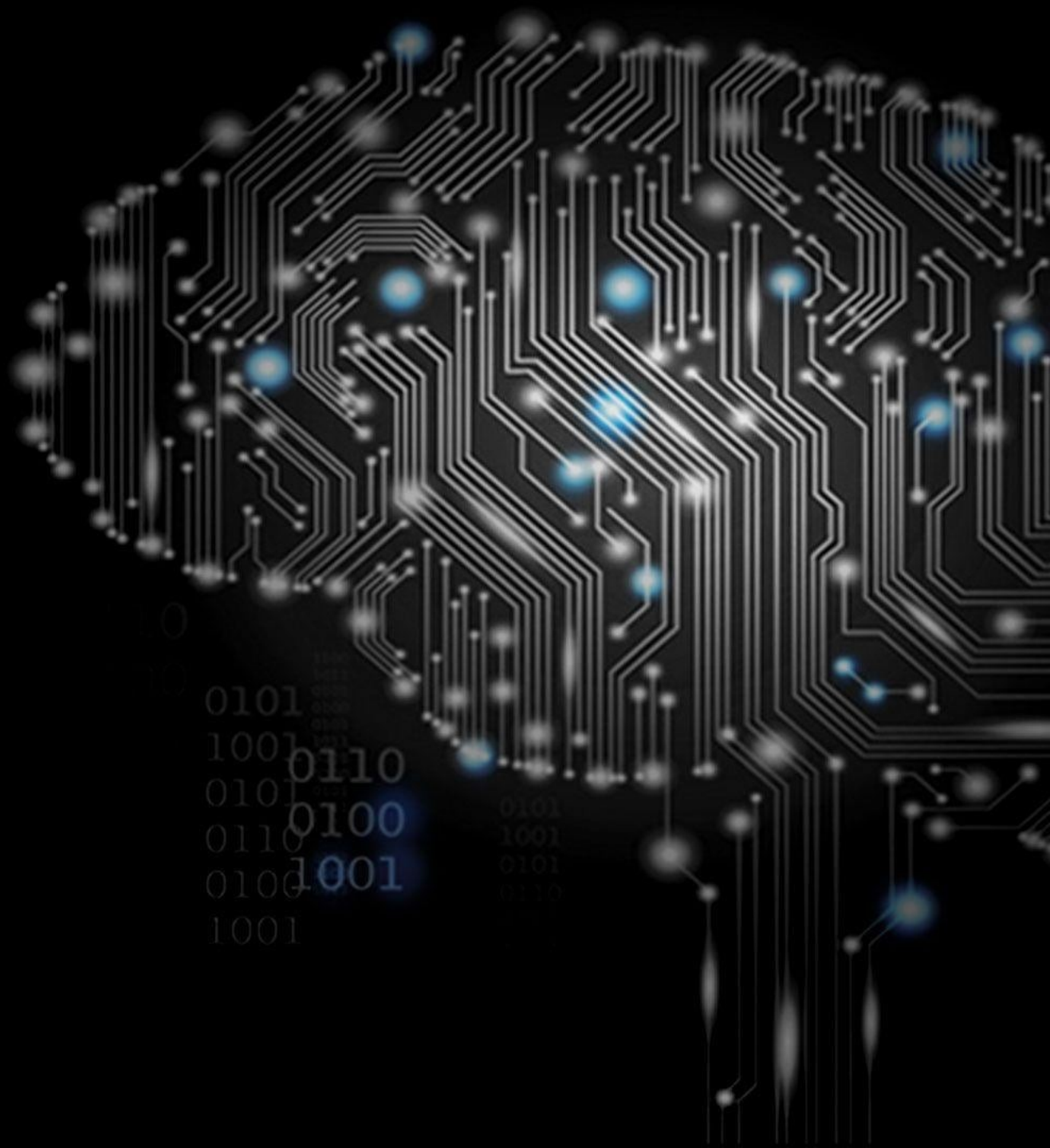
Hallucination

- A hallucination is a fluent, confident statement that is false
  - Common forms: invented facts, wrong figures, fabricated citations, non-existent API methods
  - The problem is compounded by the fact hallucinations are stated with high levels of confidence
  - OpenAI's 2025 research attributes hallucination to two causes
    - *Pretraining on next-word prediction gives no true/false signal, so arbitrary low-frequency facts (a specific date, a policy clause) cannot be learned reliably from patterns*
    - *Benchmarks that score only accuracy reward guessing over admitting uncertainty, so models are trained toward confident answers*
- Hallucination cannot be eliminated by prompting alone; it is reduced by design
  - Grounding in supplied, approved content with citations
  - Allowing and rewarding "I don't know" responses
  - Validating outputs against sources and schemas
    - *For a bank, a confident error in a customer-facing answer is a quality defect and also a conduct and compliance risk*

Design implications

- Treat the model as an unreliable, probabilistic external service
  - Apply the same resilience patterns used for any remote dependency: timeouts, retries, circuit breakers, fallbacks
  - Validate every output before it reaches business logic
  - Map to typed records, validate with Bean Validation, reject or retry on failure
  - Make every call reproducible after the fact, since it cannot be reproduced by re-running
    - *Log the prompt version, model identifier, parameters, input, output and token usage*
    - *The goal is to recreate what happened in the call since you can’t reproduce it*
- Measure quality statistically
  - Golden datasets, pass-rate thresholds and regression gates replace exact-match assertions
- Keep humans in the loop where errors are costly
  - Customer commitments, financial advice and account changes need review or hard rules





GenAI Foundations for Java Developers

2. Reasoning models and generation controls

Standard models vs. reasoning models

- Standard models generate the answer directly, token by token
  - Fast first token, lower cost, well suited to extraction, classification and rewriting
- Reasoning models generate internal reasoning tokens before the visible answer
  - Better on multi-step problems: planning, complex rules, code, analysis of long documents
  - Slower and more expensive, because reasoning tokens are billed as output
  - Reasoning content is usually not returned in full
- As of October 2026
  - The flagship models from OpenAI, Anthropic and Google are reasoning models by default
  - Some models allow reasoning to be reduced for latency-sensitive calls; others do not



What a reasoning model is

- An LLM trained to work through a problem in intermediate steps before producing its final answer
  - The intermediate steps are generated as tokens, usually called reasoning tokens or thinking tokens
  - The behaviour is produced by training, so no special prompt is needed to trigger it
- Developers of these models use reinforcement learning
  - The model attempts problems with checkable answers, such as mathematics and code, and is rewarded for correct results
  - Over many attempts, the model learns reasoning strategies that lead to correct answers more often
- Open-weight reasoning models also exist
  - This allows researchers to study how the behaviour develops



What the model learns to do

- Reinforcement learning
  - Produces recognizable problem-solving behaviours, without people writing them as rules
    - *Breaking a hard problem into simpler steps*
    - *Trying an approach, recognizing when it is not working, and switching to another*
    - *Checking intermediate results and correcting its own mistakes*
- DeepSeek's published research
  - Reported that self-reflection, verification and strategy changes emerged from reinforcement learning alone, without human-written examples of reasoning
- These behaviours explain the quality gains on tasks with many dependent steps
  - Each step can be checked before the next one builds on it
  - A standard model commits to each token as it writes the answer and cannot go back



What happens during a request

- The application sends a normal request: instructions, context and the question
  - The model generates reasoning tokens first
  - It may restate the task, list constraints, plan an approach, work through steps and check the result
  - Depending on effort and difficulty, this ranges from a few hundred to tens of thousands of tokens
  - The model then generates the visible answer, informed by the reasoning
- What the application receives
  - The final answer
  - A count of reasoning tokens in the usage data
  - On some providers, a summary of the reasoning; OpenAI does not return the raw reasoning text
- Reasoning tokens are billed as output tokens and occupy space in the context window, even though they are not shown



More thinking, better answers, up to a point

- Reasoning models introduced a second way to improve quality
  - Spending more computation when the model answers, in addition to training larger models
  - This is often called test-time compute or inference-time compute
  - OpenAI reported that o1's performance improved both with more reinforcement-learning training and with more time spent thinking at inference
- Reasoning effort is the control that exposes this trade to developers
  - Higher effort: more reasoning tokens, higher accuracy on hard problems, higher cost and latency
  - Lower effort: faster and cheaper, adequate for simpler tasks
  - The gain is not unlimited and not uniform
- On simple tasks, extra reasoning often adds cost and latency without improving the result
  - The only reliable way to choose an effort level for a use case is to measure it with an evaluation suite



Where reasoning models help, and where they do not

- Good fits:
  - Tasks with several dependent steps or rules
    - *Applying a policy with multiple conditions, for example whether a fee reversal is allowed given account type, tenure and prior reversals*
  - Analysing long documents for inconsistencies, such as comparing a contract against lending guidelines
  - Generating, reviewing and debugging code
  - Planning steps in agent workflows
- Weaker fits
  - Tasks that need speed more than deliberation
  - Simple extraction, classification and rewriting
  - Interactive chat where users expect an immediate response
  - Very high-volume processing where cost per call dominates
- Practical pattern: use low or no reasoning for routine steps, and reserve higher effort for the steps that need it

Limitations and risks

- Reasoning models still hallucinate
  - Longer reasoning can reduce some errors but does not remove the underlying causes covered earlier in this unit
  - The visible reasoning, or its summary, is not a reliable explanation of why the model answered as it did
  - Reasoning text should not be presented as the model's justification in audit or model risk documentation
- Cost and latency vary more from call to call than with standard models
- Hidden reasoning limits what can be inspected
  - Log what is available: reasoning token counts, any summaries, inputs, outputs and model version
  - Base assurance on evaluation results and guardrails, since the reasoning itself cannot be relied on



Reasoning effort is now the primary control

- Reasoning effort sets how much internal reasoning the model performs before answering
  - OpenAI defines the values none, minimal, low, medium, high, xhigh and max
    - *Each model supports only a subset*
- Examples as of October 2026
  - GPT-6 Astra rejects "none"; GPT-6.1 Sol supports neither "none" nor "minimal"; earlier GPT-5.x models default to "medium"
- Other providers expose equivalent controls under different names
  - Anthropic: an effort setting on Claude models, which can also disable extended thinking
  - Google Gemini: thinking level or thinking budget settings



Output limits and reasoning tokens

- The output limit caps everything the model generates in one call, including hidden reasoning tokens
  - If the limit is reached, the response is marked incomplete and may contain no visible answer at all
  - OpenAI returns status "incomplete" with reason "max_output_tokens"
  - Reasoning can consume from a few hundred to tens of thousands of tokens per call
    - *OpenAI recommends reserving at least 25,000 tokens for reasoning and output when starting to experiment*
- Monitor reasoning token usage per request
  - OpenAI reports it in usage.output_tokens_details.reasoning_tokens
- Engineering practice
  - Set limits from measured usage, handle the incomplete status explicitly, and alert on it



Output limits and reasoning tokens

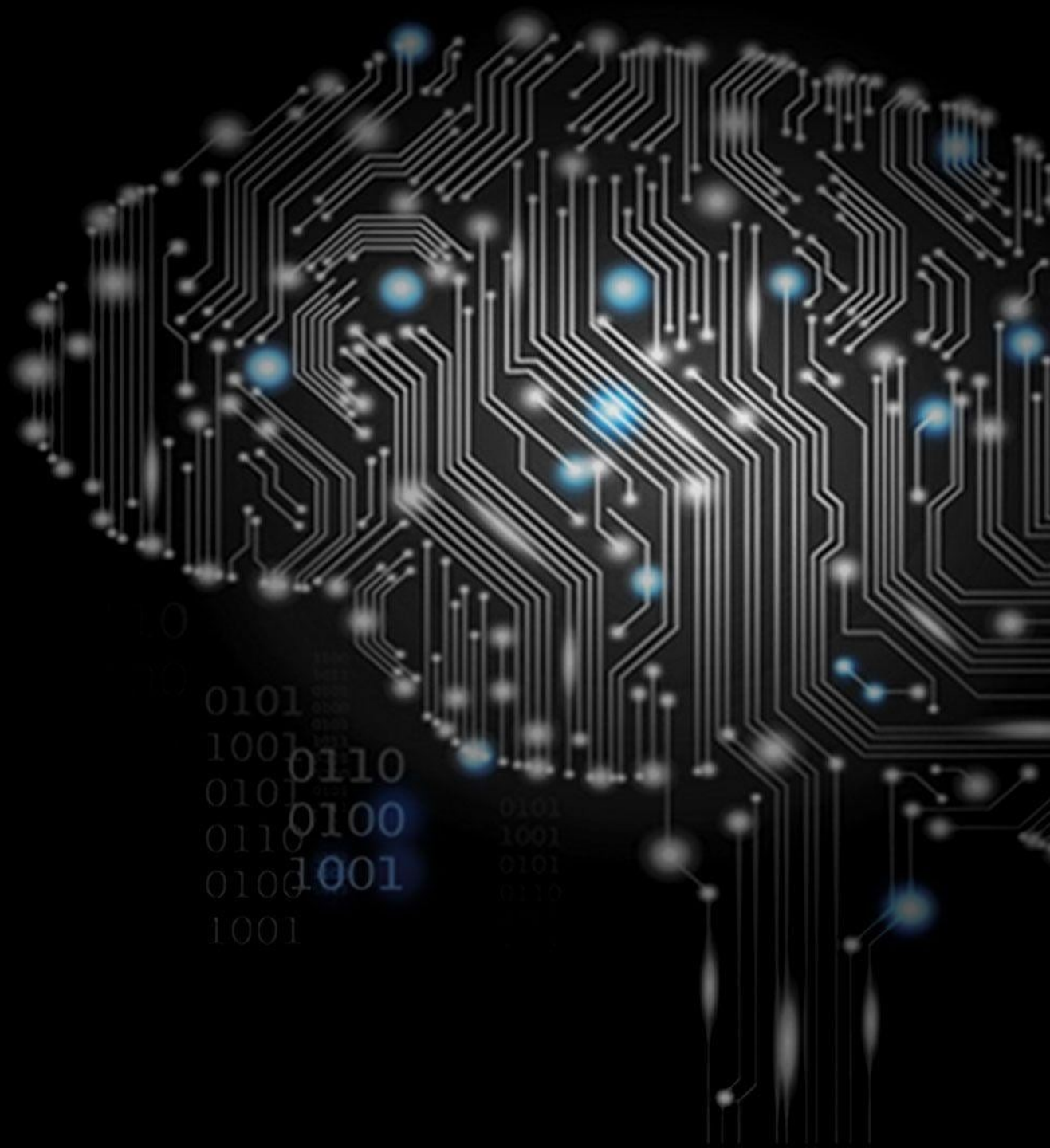
- This is a common production bug
  - A limit that worked for a standard model silently truncates a reasoning model's response, and the application receives an empty answer.
  - In Java, model "incomplete" as its own outcome type so it can never be mistaken for a successful empty answer.



What happened to temperature

- Temperature and top-p controlled randomness in token sampling on earlier models
  - Low values made output more focused; high values made it more varied
- On current reasoning models these parameters are restricted or removed
  - OpenAI GPT-6 Astra does not support custom temperature, top_p or logprobs; OpenAI's guidance is to remove them whenever reasoning effort is not "none"
  - Microsoft documents temperature, top_p, presence_penalty, frequency_penalty, logprobs and max_tokens as unsupported on reasoning models
- Providers handle unsupported values differently
  - Some reject the request with HTTP 400 (OpenAI from GPT-5.1 onward, newest Claude models)
  - Others silently ignore the value (Gemini; DeepSeek when reasoning), which is harder to detect
- What replaced temperature as the developer's control
  - Reasoning effort and verbosity for depth and length
  - Structured output schemas and examples for consistency
  - Evaluation suites for quality assurance, in place of tuning sampling settings





GenAI Foundations for Java Developers

3. Cost, latency and quality

The three-way tradeoff

- Every LLM feature balances quality, latency and cost
  - Improving one usually worsens another
  - Larger models and higher reasoning effort raise quality, latency and cost together
  - Smaller models are cheaper and faster but fail more often on complex inputs
- The right balance depends on the use case
  - Interactive customer chat: latency dominates; users notice delays of a few seconds
  - Overnight document processing: cost dominates; latency is irrelevant
  - Regulatory or credit-related decisions: quality and auditability dominate
- Make the tradeoff explicit and measurable
  - Define acceptance thresholds for each dimension before choosing a model
  - Re-evaluate when providers release new models or change prices



Where the cost comes from

- Cost per request is
  - Input tokens times input price
  - Plus output tokens times output price, with reasoning tokens counted as output
  - Example prices per million tokens, OpenAI, as of October 2026
    - *GPT-6.1 Sol: $2.00 input, $0.10 cached input, $10.00 output*
    - *GPT-6 Luna: $0.10 input, $0.01 cached input, $0.50 output*
  - Price spread between tiers is large: roughly 20 times between these two models
  - A well-tested smaller model can reduce cost by an order of magnitude for suitable tasks
- Hidden multipliers to watch
  - Resending conversation history on every turn
  - Large system prompts and few-shot examples on every call
  - Agent loops that make many model calls per user request
  - Retries after validation failures



Where latency comes from

- Time to first token
  - Network, queueing, processing the input, and any reasoning before the answer starts
  - Long prompts and high reasoning effort both increase time to first token
- Generation time
  - Output tokens are produced sequentially, so longer answers take proportionally longer
- Levers to reduce perceived and actual latency
  - Stream responses so users see text as it is generated
  - Lower reasoning effort or use a smaller model for simple steps
  - Shorten prompts and request concise output
  - Use prompt caching for long, repeated prefixes
  - Run independent calls in parallel rather than sequentially



What is a prompt cache?

- Before generating an answer, the model processes every input token and builds an internal representation of the prompt
  - For long prompts, this processing is a large share of the cost and of the time to first token
  - A prompt cache stores that processed representation for the start of the prompt (the prefix) so it can be reused
  - When a later request begins with exactly the same tokens, the provider skips reprocessing them
  - Only the new tokens after the matching prefix are processed in full
  - The cache holds the processed prompt
    - *The answer itself is still generated fresh on every call*
    - *Output, non-determinism and quality are unchanged*
  - Benefits: cached input tokens are billed at a steep discount, and responses start sooner
  - Conditions: the prefix must match exactly, it must exceed a minimum length, and the cache expires if it is not reused within a set time

Prompt caching: how it works

- Providers can cache the processed form of a repeated prompt prefix and reuse it on later requests
  - Reduces input cost substantially and reduces time to first token for long prompts
  - Caching works on the prefix only, so prompt structure matters
  - Put static content first: system instructions, policies, few-shot examples, tool definitions
  - Put variable content last: the customer message, retrieved documents, the current question
- Any change early in the prompt invalidates everything after it
  - Caches expire after a period without use, so benefit depends on traffic volume
  - Verify caching from response metadata rather than assuming it
  - OpenAI reports cached tokens in the usage details of each response



Prompt caching by provider

- OpenAI
  - Automatic for prompts longer than 1,024 tokens; no code change needed
  - Cached input is billed at a fraction of the input price
  - Longer retention options are available: a 30-minute TTL setting on GPT-6 models, and 12-hour caching announced October 20, 2026 with cache writes at 2.5 times the input price
- Anthropic (Claude, including Claude on Amazon Bedrock)
  - Enabled with a cache_control setting, either automatically at the request level or with up to four explicit breakpoints
  - Cache writes cost 1.25 times the input price for a 5-minute TTL, or 2 times for a 1-hour TTL
  - Cache reads cost 0.1 times the input price on most models, and less on the newest models
  - Minimum cacheable length ranges from 512 to 4,096 tokens depending on the model
- Pros and cons
  - Large savings for high-volume features with long, stable prompts
  - Little or no benefit for low-traffic features where caches expire between requests



Batch processing

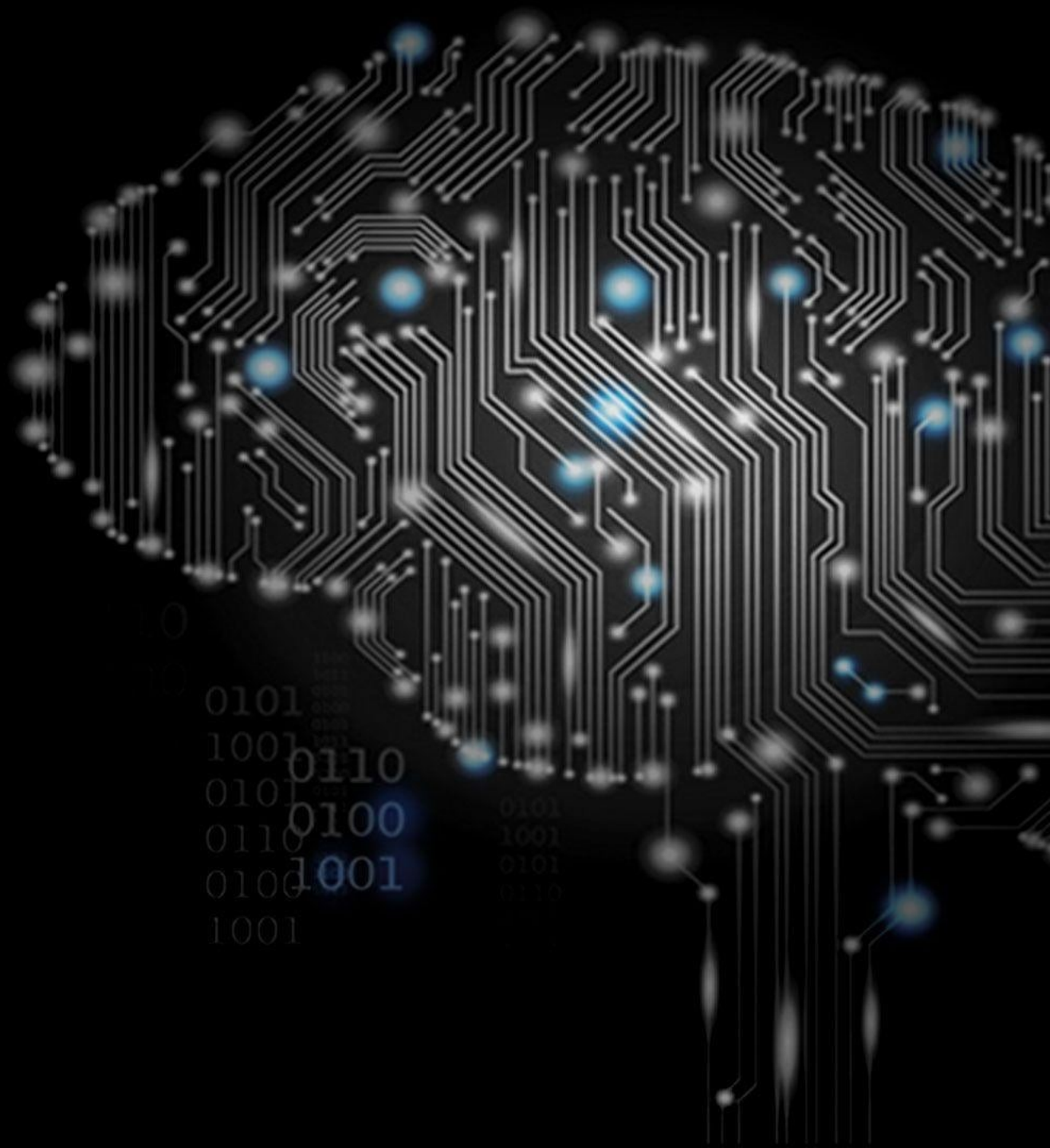
- Process large sets of requests asynchronously at a discount, in exchange for slower turnaround
  - OpenAI Batch API
    - *50 percent lower cost than synchronous calls*
    - *Results within a 24-hour completion window, often sooner*
  - Requests submitted as a JSONL file, up to 50,000 requests per batch; separate, higher rate limits
  - Supports Responses, Chat Completions, embeddings and moderation endpoints
- Bank use cases
  - Nightly classification of complaints or emails
  - Back-filling summaries or metadata for document archives
  - Running large evaluation suites against a new prompt or model version
- Poor fits
  - Anything a customer or employee is waiting on
  - Workflows where each step depends on the previous model output



Matching the model to the task

- Use the smallest model that meets the quality threshold for each step, verified by evaluation
- Model routing
  - Send simple requests to a small, fast model and escalate complex ones to a larger model
  - Routing can use rules, a classifier, or a small model acting as a router
- Mix models within one workflow
  - A small model classifies and extracts; a larger reasoning model handles the hard cases
- Pin model versions in production and upgrade deliberately
  - Provider aliases can change behaviour without a code change; dated snapshots do not
  - Re-run the evaluation suite before switching versions
- Pros and cons of multi-model designs
  - Lower average cost and latency
  - More configurations to test, monitor and govern





GenAI Foundations for Java Developers

4. The model landscape and model hosting

Frontier API models (as of October 2026)

- OpenAI GPT-6 family, released September 2026
  - GPT-6 Astra: most capable, for complex end-to-end work; tool calling requires the Responses API
  - GPT-6.1 Sol: near-Astra quality at lower cost
  - GPT-6 Luna: low-cost, high-volume tasks
- Anthropic Claude family
  - Opus 5.5 and Sonnet 5.5 as current frontier tiers; Haiku models for lower cost and latency
- Google Gemini family
  - Gemini 3.x models in Pro, Flash and Flash-Lite tiers
- Common characteristics
  - Reasoning by default, large context windows, native tool calling and structured output
  - Accessed by API; weights are not available; behaviour can change between versions

Frontier API models (as of October 2026)

- OpenAI GPT-6 family, released September 2026
  - GPT-6 Astra: most capable, for complex end-to-end work; tool calling requires the Responses API
  - GPT-6.1 Sol: near-Astra quality at lower cost
  - GPT-6 Luna: low-cost, high-volume tasks
- Anthropic Claude family
  - Opus 5.5 and Sonnet 5.5 as current frontier tiers; Haiku models for lower cost and latency
- Google Gemini family
  - Gemini 3.x models in Pro, Flash and Flash-Lite tiers
- Common characteristics
  - Reasoning by default, large context windows, native tool calling and structured output
  - Accessed by API; weights are not available; behaviour can change between versions
  - Release cadence is now measured in weeks

Open-weight models

- Open-weight models publish their trained weights, so the clients can run them on infrastructure it controls
  - Examples:
    - *Meta Llama 4, Mistral (Large 3, Small 4), Google Gemma 4, Microsoft Phi-4, OpenAI gpt-oss, Alibaba Qwen 3.x, DeepSeek V4*
  - Licences vary and need legal review
    - *Apache 2.0 or MIT for many models; custom community licences for Llama and Gemma with additional terms*
- Advantages
  - Data never leaves the client's environment; full control of versions; can be fine-tuned
  - No per-token fees, which favours very high, steady volumes
- Disadvantages
  - The client owns serving, scaling, patching and GPU capacity
  - Typically behind the best frontier models on the hardest tasks
  - Model provenance may be restricted by internal policy; some institutions limit models from certain jurisdictions
  - "Open-weight" is not the same as "open source": training data and code are usually not released



Hosting option: Amazon Bedrock

- Managed access to models from several providers through one AWS service
  - Includes Anthropic Claude, Meta Llama, Mistral, Amazon Nova, and OpenAI models
  - The Converse API provides one request and response shape across supported models
    - *Avoids model-specific request bodies; available in the AWS SDK for Java 2.x*
  - Enterprise controls inherited from AWS
    - *IAM roles instead of API keys, VPC endpoints, CloudTrail logging, regional deployment*
  - Bedrock Guardrails as a managed guardrail service (Unit 5)
- Considerations
  - Model availability varies by region; confirm what is offered in Canadian regions if data residency is required
  - Some features arrive later on Bedrock than on the provider's own API



Hosting option: Microsoft Foundry

- Microsoft's AI platform
  - Renamed from Azure AI Foundry to Microsoft Foundry at Ignite in November 2025
  - Azure OpenAI Service is now part of Foundry as "Foundry Models sold by Azure"
  - Model catalogue includes OpenAI models plus Anthropic, Meta, Mistral, xAI and Microsoft models
- Enterprise controls inherited from Azure
  - Microsoft Entra ID, managed identities, private endpoints, regional and data-zone deployments
- Considerations
  - Product and role names have changed several times; internal documentation may use older names



Hosting option: Google Gemini Enterprise Agent

- Google renamed Vertex AI to Gemini Enterprise Agent Platform on April 22, 2026
  - Model Garden offers more than 200 models
    - *Google models (Gemini, Gemma), third-party models (Anthropic Claude, Mistral) and open models (Llama)*
  - Agent Builder and Agent Engine for building and running agents
  - Enterprise controls inherited from Google Cloud
    - *IAM and service accounts, VPC Service Controls, regional endpoints*
- Considerations
  - Fewer financial institutions in Canada standardize on Google Cloud than on AWS or Azure;

Hosting option: on-premises

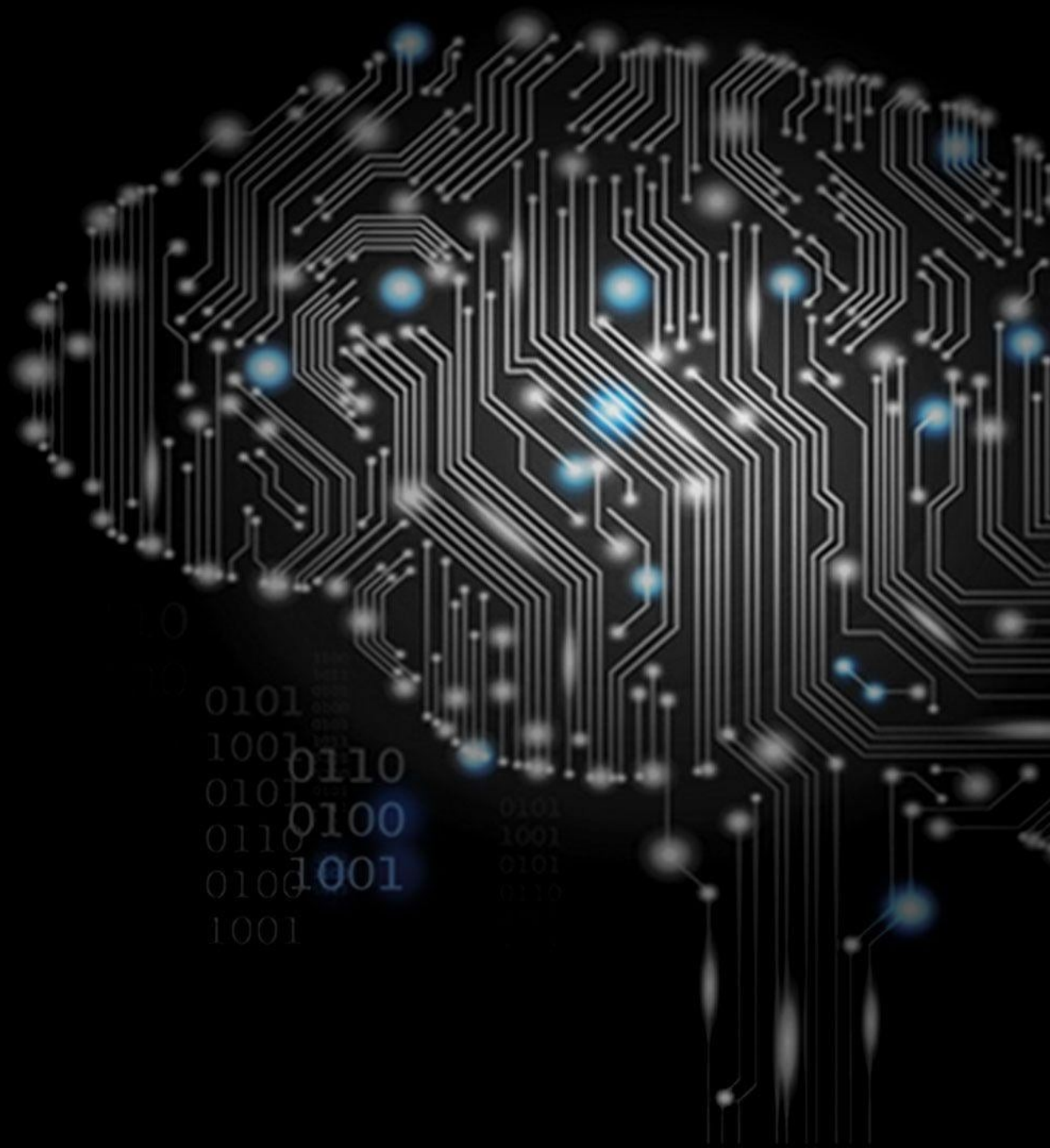
- Running open-weight models in the bank's own data centres or private cloud
  - Serving engines such as vLLM for production; Ollama for local development and testing
- Advantages
  - Maximum data control; no third-party processing of customer data
  - Predictable cost at steady high volume; no dependency on provider roadmaps
- Disadvantages
  - Requires GPU capacity, MLOps skills, capacity planning and security patching
  - Access limited to open-weight models; frontier models are not available on-premises
  - The bank must build its own guardrails, monitoring and scaling
- Common hybrid pattern
  - Sensitive workloads on-premises or in a private deployment
  - Less sensitive workloads on a managed cloud platform
  - A Java abstraction layer (Spring AI or LangChain4j) lets the same code target either



Regulatory context

- OSFI Guideline E-23, Model Risk Management, was finalized in September 2025 and takes effect in May 2027
  - Applies to models regardless of technology, explicitly including AI and machine learning
  - Principles-based and proportionate to the institution's size, complexity and risk profile
- Practical implications for teams building LLM features
  - Expect LLM applications to be inventoried, risk-rated and governed through their lifecycle
  - Validation evidence will be required: evaluation results, limitations, monitoring
  - Third-party models raise additional questions about transparency and change control
  - Data residency, privacy and third-party risk shape which hosting options are approved
- Internal model risk and AI governance teams decide what is allowed; developers supply the evidence





GenAI Foundations for Java Developers

5. The Java Ecosystem

Three levels of Java tooling

- Application frameworks
  - Spring AI and LangChain4j
    - *Provider-neutral APIs, structured output, tool calling, retrieval, memory, guardrails, observability*
- Vendor SDKs
  - openai-java, the Anthropic Java SDK, AWS SDK for Java 2.x (Bedrock Runtime)
    - *Thin, typed clients for one provider's API; earliest access to new features*
- Raw HTTP
  - Java HttpClient or Spring RestClient against the REST API
    - *Maximum control; the team owns serialization, retries and API changes*
- Most enterprise applications use a framework, and drop to a vendor SDK where a feature is not yet exposed



Spring AI 2.0

- Spring AI 2.0.0 reached general availability on June 12, 2026
  - Built for Spring Boot 4.0 and 4.1 on Spring Framework 7
- ChatClient is the primary API; requests pass through an ordered chain of advisors
  - Advisors implement cross-cutting concerns: memory, retrieval, guardrails, logging
- Built-in capabilities relevant to this course
  - ToolCallingAdvisor manages the tool-calling loop across providers
  - StructuredOutputValidationAdvisor corrects responses that fail validation
  - MCP support through the MCP Java SDK 2.0, with @McpTool, @McpResource and @McpPrompt annotations
  - Micrometer observability
- Core providers
  - OpenAI, Anthropic, Amazon Bedrock, Google GenAI, Mistral AI, DeepSeek and Ollama
- Pros and cons
  - Familiar Spring programming model, auto-configuration and Actuator integration
  - Tied to Spring Boot; 2.0 introduced breaking changes from 1.x (property names, Jackson 3)s



LangChain4j

- An independent Java library for LLM applications, not tied to any one framework
  - Integrates with Spring Boot, Quarkus and Helidon, and works in plain Java
- AI Services: declare a Java interface and LangChain4j implements it with model calls
  - Supports structured output to Java types, tools, memory and retrieval
  - Input and output guardrails as a first-class feature
- Agentic module (langchain4j-agentic) for workflow and multi-agent patterns
  - Current releases are in the 1.16.x line; the agentic module is still labelled beta
- Pros and cons
  - Framework-neutral, broad provider coverage, strong agent support
  - Separate release cadence and conventions from Spring
  - Less built-in Spring observability integration



Vendor SDKs

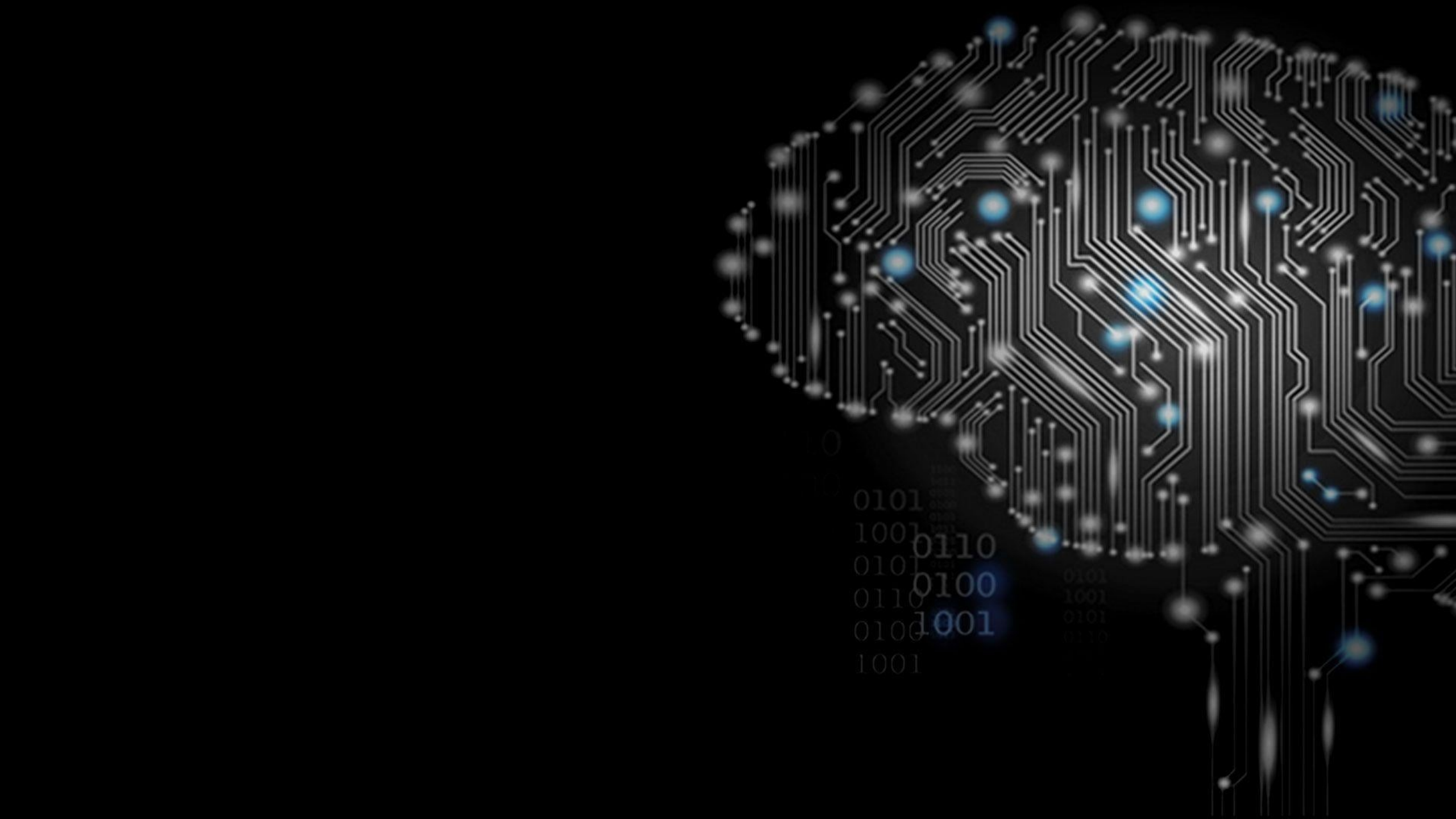
- openai-java: OpenAI's official Java SDK
  - Typed access to the Responses API and Chat Completions
  - Can also target compatible endpoints such as Microsoft Foundry
- Anthropic Java SDK: Anthropic's official client for the Claude API
  - AWS SDK for Java 2.x, Bedrock Runtime
  - The Converse API (BedrockRuntimeClient.converse) gives one request shape across Bedrock models
  - Synchronous and asynchronous (CompletableFuture) clients
- When a vendor SDK is the right choice
  - You need a provider feature before frameworks support it
  - You are building a thin integration with one provider and want minimal dependencies
- Trade-offs
  - Earliest access and full feature coverage
  - Provider lock-in; the team writes its own abstractions for retries, memory, tools and testing



How to choose

- Start from the platform the bank already runs
  - Spring Boot estate: Spring AI is the natural default
  - Quarkus or mixed estate: LangChain4j fits more naturally
- Consider the capabilities the use case needs
  - Agents and multi-agent workflows
  - Both frameworks support them; LangChain4j's agentic module offers more ready-made patterns
- Observability:
  - Spring AI integrates with Micrometer and Actuator out of the box
- Guardrails:
  - LangChain4j has first-class guardrail interfaces
  - Spring AI implements them as advisors
- Consider provider strategy
  - Multiple providers or planned switching: use a framework abstraction
  - Single provider with leading-edge features: a vendor SDK, possibly behind your own interface
  - Avoid mixing frameworks in one service
    - *Two abstractions for the same concept increase complexity and testing effort*




0101
1001
0110
0100
1001